Q-Learning Agent for Snake

A Tabular Reinforcement Learning Project

Contents

1 Problem Definition

The objective of this project is to train an autonomous agent to play the classic Snake game using **tabular Q-learning** — a model-free reinforcement learning algorithm that requires no neural networks and no prior knowledge of the environment.

The agent must learn entirely through trial and error:

- Avoid collisions with walls and its own body
- Navigate efficiently toward food
- Maximize cumulative score over time

This project serves as a foundation for understanding core RL concepts before progressing to Deep Q-Networks (DQN).

2 Environment Design

2.1 Grid and Game Rules

The environment is a 10×10 discrete grid implemented using `Pygame`. The snake starts at the center of the grid with a body of 3 cells, moving right. Food is placed randomly on any cell not occupied by the snake's body.

2.2 Snake Representation

The snake's body is represented as a `collections.deque` of `(x, y)` tuples. This data structure was chosen because movement requires:

- Adding a new head at the **front**: `appendleft()`
- Removing the tail from the **back**: `pop()`

When food is eaten, the `pop()` is skipped — the snake grows by one cell. This makes movement $O(1)$ regardless of snake length.

2.3 Movement and Actions

Actions are **relative to the current heading**, not absolute directions:

Action	Meaning
0	Turn left (relative)
1	Go straight
2	Turn right (relative)

Direction changes are computed using a clockwise rotation list:

```
clockwise = [(1,0), (0,1), (-1,0), (0,-1)]
#            RIGHT   DOWN   LEFT    UP

idx = clockwise.index(self.direction)
new_direction = clockwise[(idx + delta) % 4]
# delta: -1=left, 0=straight, +1=right
```

The modulo operation ensures wrapping around the list without index errors.

2.4 Timeout Mechanism

To prevent the agent from looping indefinitely without eating food, a frame counter terminates the episode if:

$$\text{frame\_count} > 100 \times \text{len(snake)} \tag{1}$$

The threshold scales with snake length — a longer snake legitimately needs more steps to navigate around its own body.

3 State Representation

3.1 Design Philosophy

Using the full 10×10 grid as the state would produce millions of possible state combinations, making tabular Q-learning computationally intractable. Instead, a **compact 11-bit binary state vector** was designed through feature engineering:

$$s = [\underbrace{d_s,\, d_l,\, d_r}_{\text{danger}},\ \underbrace{f_l,\, f_r,\, f_u,\, f_d}_{\text{food direction}},\ \underbrace{m_l,\, m_r,\, m_u,\, m_d}_{\text{movement}}] \tag{2}$$

This limits the state space to at most $2^{11} = 2048$ possible states.

3.2 Danger Flags (3 bits)

Danger is detected by simulating one step in each relative direction and checking for collision:

```
dir_straight = clockwise[idx % 4]
dir_left     = clockwise[(idx - 1) % 4]
dir_right    = clockwise[(idx + 1) % 4]

danger_straight = int(game.is_collision(straight))
danger_left     = int(game.is_collision(left))
danger_right    = int(game.is_collision(right))
```

3.3 Food Direction Flags (4 bits)

Food direction is encoded as 4 absolute binary flags. Actual coordinates were avoided to prevent state space explosion:

```
food_left  = int(food[0] < head[0])
food_right = int(food[0] > head[0])
food_up    = int(food[1] < head[1])   # y increases downward
food_down  = int(food[1] > head[1])
```

Note: In Pygame the y-axis is inverted — y increases downward.

3.4 Movement Direction Flags (4 bits)

Current heading is encoded as 4 mutually exclusive binary flags:

```
dir_right = int(game.direction == (1,  0))
dir_left  = int(game.direction == (-1, 0))
dir_down  = int(game.direction == (0,  1))
dir_up    = int(game.direction == (0, -1))
```

Exactly one of these flags is 1 at any time.

4 Reward Design

The reward function was designed to shape three specific behaviors:

Event	Reward	Purpose
Food eaten	$+10$	Incentivize objective
Collision	-10	Penalize death
Each step	-0.1	Force efficiency

The step penalty is critical — without it, the agent has no incentive to find food efficiently and may survive by looping indefinitely without ever targeting food.

5 Q-Learning Algorithm

5.1 The Q-Table

The Q-table is implemented as a Python dictionary mapping state tuples to lists of 3 action values:

```
q_table = {}
# Example entry:
# (1,0,0,1,0,0,1,0,0,1,0) -> [Q_left, Q_straight, Q_right]
```

Tuples are used as keys because Python dictionary keys must be **immutable** (hashable). Lists cannot serve as keys.

Unseen states default to $[0, 0, 0]$ — no prior preference.

5.2 The Bellman Equation

After every step, the Q-table is updated using the Bellman equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{3}$$

Where:

- $\alpha = 0.1$: learning rate — how much new information overwrites old
- $\gamma = 0.9$: discount factor — how much future rewards are valued
- r: immediate reward received
- $\max_{a'} Q(s',a')$: best possible value from the next state
- $r + \gamma \max_{a'} Q(s',a') - Q(s,a)$: the **Bellman error** — analogous to prediction error in gradient descent

5.3 Epsilon-Greedy Exploration

The agent balances exploration and exploitation using an ε-greedy policy:

$$\text{action} = \begin{cases} \text{random} & \text{if } u < \varepsilon \\ \arg\max_a Q(s,a) & \text{otherwise} \end{cases} \qquad u \sim \mathcal{U}(0,1) \tag{4}$$

ε decays exponentially after each episode:

$$\varepsilon \leftarrow \max(\varepsilon \times 0.995,\ 0.01) \tag{5}$$

Exponential decay was chosen over linear decay because it decreases rapidly at first (when exploration is most needed) and slows down as the agent becomes more competent. The floor of 0.01 ensures the agent never becomes fully greedy — preserving a 1% chance of discovering better paths.

6 Project Architecture

The project follows a modular architecture with clear separation of concerns:

Module	Responsibility
`game.py`	Environment, rules, collision, rewards
`agent.py`	Q-table, state, action selection, updates
`train.py`	Episode loop, metrics tracking
`utils.py`	Learning curve visualization

Each component was verified independently before integration — following the principle of testing components in isolation before combining them.

7 Results and Analysis

7.1 Training Configuration

Parameter	Value
Episodes	3000
Grid size	10×10
Learning rate α	0.1
Discount γ	0.9
Initial ε	1.0
Final ε	0.01
Decay rate	0.995 per episode

7.2 Learning Curve

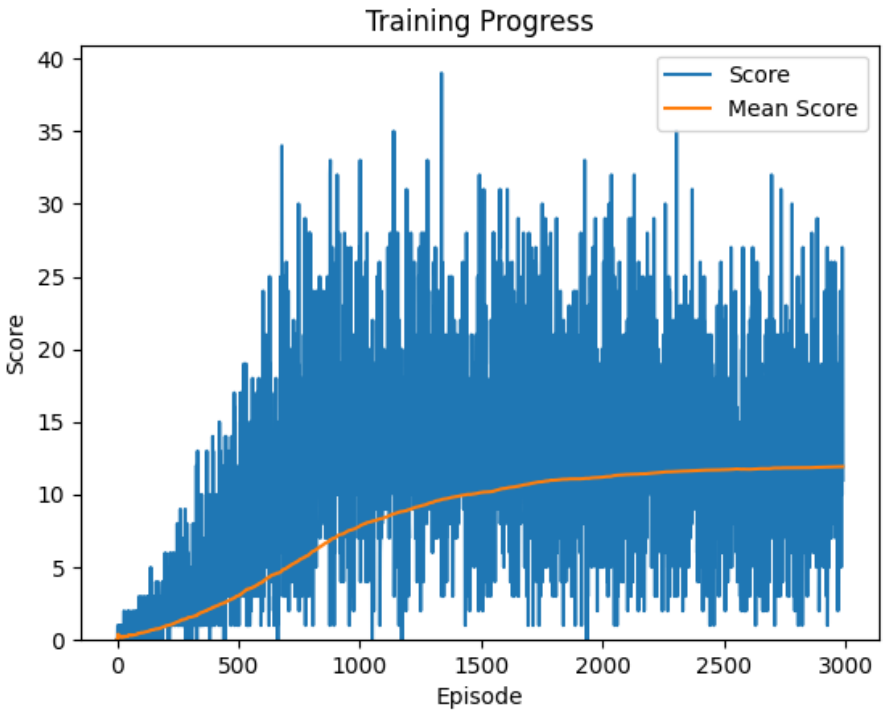

Figure 1: Training progress over 3000 episodes. Blue: score per episode. Orange: cumulative mean score.

7.3 Three Phases of Learning

The learning curve reveals three distinct phases:

Phase 1 — Pure Exploration (episodes 0–200): $\varepsilon \approx 1.0$. The agent moves randomly. The Q-table is being populated with initial experience but scores remain near zero. The agent has not yet learned to avoid walls.

Phase 2 — Mixed Strategy (episodes 200–1500): ε decays from 1.0 toward 0.01. The agent first learns to avoid walls (surviving longer), then begins targeting food. Mean score rises steeply. Occasional high scores appear as the agent discovers efficient paths.

Phase 3 — Greedy Exploitation (episodes 1500–3000): $\varepsilon \approx 0.01$. The Q-table has converged. The agent consistently targets food and avoids death. Mean score plateaus around 11–12. Best score reached **39**.

7.4 Final Metrics

Metric	Value
Best score	39
Final mean score	11.93
Final ε	0.010

8 Limitations and Upgrade Path

8.1 The Ceiling Problem

The mean score plateaus at approximately 12 and does not improve further. This ceiling is a **fundamental limitation of tabular Q-learning** caused by *state aliasing* — two different real game situations that produce identical 11-bit state vectors.

For example, the state `(0,0,0,1,0,0,0,0,1,0,0)` means:

> *"No danger anywhere, food is to the left, moving right"*

But this tells the agent nothing about what lies 3–4 steps ahead. A dead end and an open corridor look identical. The Q-table cannot learn to distinguish them.

8.2 Upgrade Path: Deep Q-Network (DQN)

The natural next step is to replace the Q-table with a neural network:

Component	Upgrade
Q-table	Neural network (PyTorch)
11-bit state	Full grid encoding (100 cells)
Direct update	Experience replay buffer
Single network	Target network (stability)

A neural network eliminates state aliasing by accepting the full grid as input — the agent can now distinguish situations that looked identical under the 11-bit encoding. Experience replay breaks the correlation between consecutive updates, and a target network stabilizes training.

The recommended next environment for DQN is **Flappy Bird** — a continuous state space that makes tabular Q-learning impossible, forcing the use of function approximation from the start.

9 Conclusion

This project demonstrated that a tabular Q-learning agent can learn non-trivial behavior in a discrete environment without any neural networks, supervised labels, or hardcoded rules. Starting from purely random movement, the agent reached a best score of 39 and a stable mean of 11.93 over 3000 episodes — entirely through the Bellman update rule and epsilon-greedy exploration.

The plateau at mean score 12 is not a failure — it is **evidence of the method's boundary conditions**, and the clearest possible motivation for the next step: Deep Q-Networks.